# Informe Técnico: Simulador de Datos Sísmicos (SEG-Y) y Optimización de E/S Mediante Estructuras de Datos y Prefetching

**Jose Alvarez**
C.I.: 31389258

**Jose Nuñez**
C.I.: 31581050

*Universidad de Carabobo*
*Facultad de Ciencias y Tecnología*
*Departamento de Computación – Arquitectura del Computador*

Octubre 2026

**Resumen**

En este informe se presenta el desarrollo, implementación y análisis de rendimiento de un simulador de datos sísmicos en formato SEG-Y desarrollado en C++17. Se abordan los desafíos asociados al manejo de volúmenes de datos masivos en geofísica, evaluando diferentes backends de almacenamiento en memoria, indexación espacial basada en árboles rojinegros y estrategias de carga anticipada (*prefetching*) con caché LRU para mitigar los cuellos de botella de Entrada/Salida (E/S). Los resultados demuestran que el uso de prefetching espacial incrementa la tasa de aciertos (*hit ratio*) al 100 % y reduce drásticamente las operaciones de lectura física.

## 1. El Problema

En la industria geofísica y de exploración petrolera, los conjuntos de datos sísmicos tridimensionales (3D) representan volúmenes masivos de información estructurada que frecuentemente alcanzan gigabytes o terabytes. Un archivo en formato estándar *SEG-Y* contiene cabeceras globales y miles de trazas individuales (registros de ondas reflejadas en el subsuelo).

El problema fundamental radica en que los patrones de acceso durante la interpretación interactiva o el procesamiento sísmico suelen ser complejos (secuenciales, inversos, aleatorios o con saltos espaciales). Cuando un software accede a estos volúmenes sin una estrategia adecuada de gestión de memoria y caché, se producen severos cuellos de botella de Entrada/Salida (E/S) y una saturación ineficiente de la memoria RAM. El objetivo principal de este proyecto es construir un banco de pruebas que permita simular, medir y comparar el impacto de distintas arquitecturas de software y algoritmos de optimización sobre un volumen sísmico sintético.

## 2. Técnicas Computacionales para su Solución

Para resolver los desafíos de rendimiento y almacenamiento, el simulador se estructuró de manera modular en C++17, implementando los siguientes componentes algorítmicos y estructuras de datos:

### 2.1. Parser del Formato SEG-Y (`SEGYParser`)

El sistema implementa un analizador robusto capaz de leer y procesar la cabecera binaria de 400 bytes (para extraer parámetros como el intervalo de muestreo temporal $\Delta t$ y el formato

de codificación de las muestras) y recorrer iterativamente cada traza con su respectiva cabecera de 240 bytes. Asimismo, incluye un generador sintético basado en ondas *Ricker* correlacionadas espacialmente que permite crear cubos de prueba configurables (por ejemplo, dimensiones de $64 \times 64$ o $128 \times 128$ trazas con 256/512 muestras por traza).

### 2.2. Backends de Almacenamiento en Memoria (`StorageBackend`)

Se diseñó una interfaz abstracta para comparar dos estructuras de contenedores de la STL:

- `VectorStorage` **(std::vector):** Almacena las trazas en un bloque de memoria contiguo, maximizando la localidad espacial y optimizando los accesos aleatorios, a costo de posibles reasignaciones dinámicas si el tamaño varía.
- `DequeStorage` **(std::deque):** Utiliza bloques de memoria fragmentados pero encadenados, ofreciendo un rendimiento eficiente en inserciones masivas durante la carga inicial del archivo.

### 2.3. Indexación Espacial (`TraceIndex`)

En geofísica, las trazas no se consultan por su índice físico secuencial, sino por sus coordenadas lógicas en la malla de adquisición: el par de coordenadas *(inline, crossline)*. Para resolver consultas espaciales en tiempo óptimo, se implementó un índice basado en un árbol rojo-negro a través de std::map. Este índice relaciona la clave lógica *(inline, crossline)* con la posición física exacta del vector de almacenamiento en un tiempo logarítmico $\mathcal{O}(\log n)$.

### 2.4. Gestión de Caché y Prefetching (`Prefetcher`)

Para evitar accesos repetitivos al almacenamiento, se desarrolló un sistema de caché con política de reemplazo *LRU* (*Least Recently Used*) acoplado a un algoritmo de *prefetching* espacial. Cuando una traza $i$ es solicitada, el prefetcher predice y carga de forma proactiva un número configurable de trazas vecinas espaciales $(i+1, i+2, \ldots, i+k)$, anticipándose al comportamiento de lectura del usuario y reduciendo la latencia de E/S.

## 3. Análisis y Comparación de Resultados

El simulador fue ejecutado sobre un archivo SEG-Y sintético de prueba ($\sim$ 4,94 MB, compuesto por 4096 trazas de 256 muestras cada una). A continuación se analizan los resultados empíricos obtenidos en el banco de pruebas:

### 3.1. Rendimiento de Carga y Uso de Memoria

La carga del volumen completo demostró ser sumamente ligera en ambas estructuras de la STL, operando con un consumo de memoria RAM aproximado de 4,25 MB:

- **std::vector:** Tiempo de carga de 57,0 ms.
- **std::deque:** Tiempo de carga de 70,0 ms.

Por su parte, la construcción del árbol rojo-negro para indexar las 4096 trazas tomó apenas 1,0 ms, consumiendo una huella de memoria auxiliar de 192,00 KB.

### 3.2. Impacto del Prefetching en Patrones de Acceso

La evaluación bajo diferentes patrones de consulta (secuencial, inverso, aleatorio y con saltos) evidenció el comportamiento crítico de la caché. Sin prefetching, el *Hit Ratio* se mantiene en 0,000, obligando al sistema a realizar la totalidad de las 4096 cargas físicas al backend en cada recorrido.

Sin embargo, al activar el mecanismo de *prefetching* con profundidades de 2, 4 u 8 vecinos bajo un patrón secuencial, se observa el impacto analítico clave del proyecto:

Cuadro 1: Rendimiento del sistema bajo diferentes niveles de Prefetching (Patrón Secuencial).

| **Prefetch (Vecinos)** | **Tiempo (ms)** | **Hit Ratio** | **Cargas Físicas** | **Prefetches** |
|---|---|---|---|---|
| off (Sin caché) | 2,0 | 0,000 | 4096 | – |
| 2 vecinos | 10,0 | 1,000 | 1 | – |
| 4 vecinos | 5,0 | 1,000 | 1 | – |
| 8 vecinos | 4,0 | 1,000 | 1 | – |

Como se aprecia en la Tabla 1, al habilitar la precarga espacial, la tasa de aciertos (*Hit Ratio*) salta inmediatamente a 1,000 **(100 % de aciertos)** y el número de accesos físicos al almacenamiento se desploma drásticamente de 4096 operaciones a **tan solo 1 carga inicial**. Esto demuestra empíricamente que la predicción basada en la localidad espacial mitiga de forma radical los cuellos de botella en el procesamiento de datos sísmicos.

## 4. Conclusiones

El desarrollo e implementación de este simulador permitió comprobar que la elección adecuada de estructuras de datos contiguas (std::vector), combinada con una indexación espacial eficiente mediante árboles rojinegros y políticas de caché con *prefetching*, optimiza de manera exponencial el rendimiento en la lectura de cubos sísmicos SEG-Y. El proyecto cumple a cabalidad con los requerimientos de arquitectura de alto rendimiento y provee una base sólida para futuras extensiones en entornos distribuidos o de almacenamiento en disco secundario.